

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Cryptography and Network Security

COURSE CODE: CSA51

COURSE OUTCOME COVERED

CO3: Examine cryptographic hash algorithms, digital signature schemes, and assess Kerberos and X.509 authentication frameworks for ensuring integrity, authentication, and non-repudiation. CO (BL4, BL5)

Assessment Tool Weightage: 40%

QUESTIONS: 5

TOTAL MARKS: 25

EXPERIMENTS

Code of Conduct:

I **RITHISH KUMAR D** (Name / Reg No: **192565089**) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: *Rithish Kumar D*

Annexure A

Experiments

1. Design and implement a program in C/Python to generate and compare hashes using MD5 and SHA-256 for a given dataset. Analyze the differences in collision resistance and performance, and evaluate the suitability of each algorithm for secure applications.
2. Develop a C/Python application to simulate secure hashing (SHA family) for file integrity verification. Analyze how hash values change with minor modifications in input and evaluate the effectiveness of the algorithm in detecting tampering.
3. Implement a digital signature system in C/Python using public-key cryptography (e.g., RSA). Analyze the process of signing and verification, and evaluate how the system ensures authentication and non-repudiation in a real-world scenario.
4. Create a simplified Kerberos authentication simulation in C/Python that includes ticket generation, distribution, and validation. Analyze the workflow and evaluate how replay attacks are prevented in your implementation.
5. Develop a C/Python program to parse and validate X.509 certificates. Analyze certificate fields (issuer, validity, public key) and evaluate the trustworthiness of certificates based on validation checks.

Question 1

Design and implement a program in C/Python to generate and compare hashes using MD5 and SHA-256 for a given dataset. Analyze the differences in collision resistance and performance, and evaluate the suitability of each algorithm for secure applications.

Program (Python):

```
"""
Experiment 1: Generate and compare MD5 and SHA-256 hashes for a given dataset.
Analyze collision resistance and performance, and evaluate suitability
of each algorithm for secure applications.
"""

import hashlib
import time
import os

def generate_hashes(data: bytes):
    md5_hash = hashlib.md5(data).hexdigest()
    sha256_hash = hashlib.sha256(data).hexdigest()
    return md5_hash, sha256_hash

def measure_performance(data: bytes, iterations: int = 20000):
    start = time.perf_counter()
    for _ in range(iterations):
        hashlib.md5(data).hexdigest()
    md5_time = time.perf_counter() - start

    start = time.perf_counter()
    for _ in range(iterations):
        hashlib.sha256(data).hexdigest()
    sha_time = time.perf_counter() - start

    return md5_time, sha_time

def demonstrate_collision_sensitivity(data: bytes):
    """Flip a single bit and show how drastically the hash changes
    (avalanche effect) for both algorithms."""
    tampered = bytearray(data)
    tampered[0] ^= 0x01 # flip the lowest bit of the first byte
    tampered = bytes(tampered)

    original_md5, original_sha = generate_hashes(data)
    tampered_md5, tampered_sha = generate_hashes(tampered)

    return (original_md5, tampered_md5), (original_sha, tampered_sha)

if __name__ == "__main__":
    dataset = b"SIMATS Engineering - Cryptography and Network Security Dataset 2026"

    print("=" * 70)
    print("DATASET:", dataset.decode())
    print("=" * 70)

    md5_val, sha256_val = generate_hashes(dataset)
    print(f"\nMD5 Hash      : {md5_val}")
    print(f"Length          : {len(md5_val) * 4} bits")
    print(f"\nSHA-256 Hash: {sha256_val}")
    print(f"Length          : {len(sha256_val) * 4} bits")

    print("\n" + "-" * 70)
    print("PERFORMANCE COMPARISON (20,000 iterations)")
    print("-" * 70)
    md5_time, sha_time = measure_performance(dataset)
    print(f"MD5      total time : {md5_time:.4f} s")
    print(f"SHA-256 total time: {sha_time:.4f} s")
    print(f"SHA-256 is {sha_time / md5_time:.2f}x the time cost of MD5")

    print("\n" + "-" * 70)
    print("AVALANCHE EFFECT (1 bit flipped in input)")
    print("-" * 70)
    (o_md5, t_md5), (o_sha, t_sha) = demonstrate_collision_sensitivity(dataset)
    print(f"Original MD5 : {o_md5}")
```

```

print(f"Tampered MD5 : {t_md5}")
diff_bits_md5 = bin(int(o_md5, 16) ^ int(t_md5, 16)).count("1")
print(f"Bits changed : {diff_bits_md5} / 128")

print(f"\nOriginal SHA-256 : {o_sha}")
print(f"Tampered SHA-256 : {t_sha}")
diff_bits_sha = bin(int(o_sha, 16) ^ int(t_sha, 16)).count("1")
print(f"Bits changed      : {diff_bits_sha} / 256")

print("\n" + "=" * 70)
print("ANALYSIS")
print("=" * 70)
print("""
1. Collision Resistance:
- MD5 (128-bit digest) is cryptographically BROKEN. Practical collision
  attacks (e.g., chosen-prefix collisions) have been demonstrated since
  2004-2008, making it unsuitable for any security-critical use.
- SHA-256 (256-bit digest, part of SHA-2) has no known practical
  collision or pre-image attacks and remains secure for real-world use.

2. Performance:
- In theory MD5 (128-bit state, simpler round function) should be
  cheaper per byte than SHA-256 (256-bit state, more complex message
  schedule). In practice, on modern CPUs many hashlib builds use
  hardware-accelerated SHA extensions, so the measured timings above
  can even show SHA-256 matching or beating MD5.
- Either way, the difference is small enough that raw speed should
  never be the deciding factor between the two algorithms.

3. Suitability:
- MD5: acceptable ONLY for non-security use such as quick checksums
  for accidental data-corruption detection (e.g., file de-duplication),
  NEVER for passwords, digital signatures, or integrity verification
  against an active adversary.
- SHA-256: suitable for password hashing (with salting/KDF), digital
  signatures, blockchain, TLS certificates, and any application
  requiring strong collision and pre-image resistance.
""")

```

Output:

```

=====
DATASET: SIMATS Engineering - Cryptography and Network Security Dataset 2026
=====

MD5 Hash       : e950e5b4500ca31b60e3ba17a8ecb346
Length         : 128 bits

SHA-256 Hash   : 4a65bc4a99febe2ce229fb95e2bc5128365f1244b6ba6e087bc6e687ee53c0fc
Length         : 256 bits

-----
PERFORMANCE COMPARISON (20,000 iterations)
-----
MD5      total time : 0.0139 s
SHA-256 total time: 0.0177 s
SHA-256 is 1.28x the time cost of MD5

-----
AVALANCHE EFFECT (1 bit flipped in input)
-----
Original MD5 : e950e5b4500ca31b60e3ba17a8ecb346
Tampered MD5 : 91d9e1cb3cda663d0968c6c3c53aa9f1
Bits changed  : 67 / 128

Original SHA-256 : 4a65bc4a99febe2ce229fb95e2bc5128365f1244b6ba6e087bc6e687ee53c0fc
Tampered SHA-256 : f557e14f4ba1544eecbf16c7fc93620845228dfb5c5ba1dcbee194fec29339fe
Bits changed     : 137 / 256

=====
ANALYSIS
=====

```

1. Collision Resistance:
 - MD5 (128-bit digest) is cryptographically BROKEN. Practical collision attacks (e.g., chosen-prefix collisions) have been demonstrated since 2004-2008, making it unsuitable for any security-critical use.
 - SHA-256 (256-bit digest, part of SHA-2) has no known practical collision or pre-image attacks and remains secure for real-world use.

2. Performance:

- In theory MD5 (128-bit state, simpler round function) should be cheaper per byte than SHA-256 (256-bit state, more complex message schedule). In practice, on modern CPUs many hashlib builds use hardware-accelerated SHA extensions, so the measured timings above can even show SHA-256 matching or beating MD5.
- Either way, the difference is small enough that raw speed should never be the deciding factor between the two algorithms.

3. Suitability:

- MD5: acceptable ONLY for non-security use such as quick checksums for accidental data-corruption detection (e.g., file de-duplication), NEVER for passwords, digital signatures, or integrity verification against an active adversary.
- SHA-256: suitable for password hashing (with salting/KDF), digital signatures, blockchain, TLS certificates, and any application requiring strong collision and pre-image resistance.

Question 2

Develop a C/Python application to simulate secure hashing (SHA family) for file integrity verification. Analyze how hash values change with minor modifications in input and evaluate the effectiveness of the algorithm in detecting tampering.

Program (Python):

```
"""
Experiment 2: Simulate secure hashing (SHA family) for file integrity
verification. Show how hash values change with minor input modification
and evaluate effectiveness in detecting tampering.
"""

import hashlib
import os

def compute_file_hash(filepath: str, algorithm: str = "sha256") -> str:
    h = hashlib.new(algorithm)
    with open(filepath, "rb") as f:
        for chunk in iter(lambda: f.read(4096), b''):
            h.update(chunk)
    return h.hexdigest()

def verify_integrity(filepath: str, reference_hash: str, algorithm: str = "sha256") -> bool:
    current_hash = compute_file_hash(filepath, algorithm)
    return current_hash == reference_hash

if __name__ == "__main__":
    original_path = "original_file.txt"
    tampered_path = "tampered_file.txt"

    original_content = (
        "Confidential Report - Q3 Financial Summary\n"
        "Total Revenue: $1,250,000\n"
        "Approved by: Finance Department\n"
    )

    with open(original_path, "w") as f:
        f.write(original_content)

    print("=" * 70)
    print("STEP 1: Compute baseline hashes for the original file")
    print("=" * 70)
    for algo in ["sha1", "sha256", "sha512"]:
        digest = compute_file_hash(original_path, algo)
        print(f"{algo.upper():8s}: {digest}")

    baseline_sha256 = compute_file_hash(original_path, "sha256")

    print("\n" + "=" * 70)
    print("STEP 2: Verify integrity of an UNCHANGED file")
    print("=" * 70)
    result = verify_integrity(original_path, baseline_sha256, "sha256")
    print(f"File unchanged -> Integrity check passed: {result}")

    # Introduce a minor modification (single character change)
    tampered_content = original_content.replace("1,250,000", "1,259,000")
    with open(tampered_path, "w") as f:
        f.write(tampered_content)

    print("\n" + "=" * 70)
    print("STEP 3: Verify integrity after a MINOR modification")
    print("(changed '1,250,000' to '1,259,000' - a single digit)")
    print("=" * 70)
    tampered_sha256 = compute_file_hash(tampered_path, "sha256")
    print(f"Original SHA-256 : {baseline_sha256}")
    print(f"Tampered SHA-256 : {tampered_sha256}")

    result = verify_integrity(tampered_path, baseline_sha256, "sha256")
    print(f"\nIntegrity check against original hash passed: {result}")

    diff_bits = bin(int(baseline_sha256, 16) ^ int(tampered_sha256, 16)).count("1")
    print(f"Number of differing bits between digests: {diff_bits} / 256 "
```

```

f"({diff_bits/256*100:.1f}% of bits changed)")

# cleanup
os.remove(original_path)
os.remove(tampered_path)

print("\n" + "=" * 70)
print("ANALYSIS")
print("=" * 70)
print("""
1. Even a single-character change in a multi-line file produces a
completely different digest (the avalanche effect), which is exactly
the property that makes cryptographic hashes useful for tamper
detection: an attacker cannot make a small, unnoticed change without
the hash changing.

2. Integrity verification workflow used here:
    a. Compute and store a trusted reference hash when the file is known
       to be authentic (e.g., at time of upload/signing).
    b. Before trusting the file again, recompute its hash and compare it
       to the stored reference.
    c. Any mismatch means the file has been altered (accidentally or
       maliciously) since the reference hash was taken.

3. Effectiveness: SHA-256/SHA-512 are effective for this purpose because
they are pre-image and collision resistant - an adversary cannot feasibly
craft a different file that produces the same hash, so a hash match is
strong evidence the file is unmodified. SHA-1 is included above only
for comparison; it is deprecated for security purposes because
practical collision attacks exist, so SHA-256 or stronger should be
used in real integrity-verification systems.
""")

```

Output:

```

=====
STEP 1: Compute baseline hashes for the original file
=====
SHA1      : 16c92f5ac3925b8b6dee92be193970bb453eac04
SHA256    : 6565df3810d108232bb63705cc5cab58b56a8ba9d295b9f917ebc8a91f409c1b
SHA512    : 84fccaf7f7de5ad9e12354c3f3482e249d1310747a4291ad2dfd7b2a0e29ef5c9844fa165ed6130a5520ad81c6aa973c60313d40bd77f52

=====
STEP 2: Verify integrity of an UNCHANGED file
=====
File unchanged -> Integrity check passed: True

=====
STEP 3: Verify integrity after a MINOR modification
(changed '1,250,000' to '1,259,000' - a single digit)
=====
Original SHA-256 : 6565df3810d108232bb63705cc5cab58b56a8ba9d295b9f917ebc8a91f409c1b
Tampered SHA-256 : 9b38ec05c629c890b7aeaea8886de60c910e997842d5e8ced26b499fb83c3b2c

Integrity check against original hash passed: False
Number of differing bits between digests: 118 / 256 (46.1% of bits changed)

=====
ANALYSIS
=====

1. Even a single-character change in a multi-line file produces a
completely different digest (the avalanche effect), which is exactly
the property that makes cryptographic hashes useful for tamper
detection: an attacker cannot make a small, unnoticed change without
the hash changing.

2. Integrity verification workflow used here:
    a. Compute and store a trusted reference hash when the file is known
       to be authentic (e.g., at time of upload/signing).
    b. Before trusting the file again, recompute its hash and compare it
       to the stored reference.
    c. Any mismatch means the file has been altered (accidentally or
       maliciously) since the reference hash was taken.

3. Effectiveness: SHA-256/SHA-512 are effective for this purpose because
they are pre-image and collision resistant - an adversary cannot feasibly
craft a different file that produces the same hash, so a hash match is
strong evidence the file is unmodified. SHA-1 is included above only

```

for comparison; it is deprecated for security purposes because practical collision attacks exist, so SHA-256 or stronger should be used in real integrity-verification systems.

Question 3

Implement a digital signature system in C/Python using public-key cryptography (e.g., RSA). Analyze the process of signing and verification, and evaluate how the system ensures authentication and non-repudiation in a real-world scenario.

Program (Python):

```
"""
Experiment 3: Digital signature system using RSA public-key cryptography.
Demonstrates signing and verification, and how it provides
authentication and non-repudiation.
"""

from cryptography.hazmat.primitives.asymmetric import rsa, padding
from cryptography.hazmat.primitives import hashes, serialization

def generate_keypair(key_size: int = 2048):
    private_key = rsa.generate_private_key(public_exponent=65537, key_size=key_size)
    public_key = private_key.public_key()
    return private_key, public_key

def sign_message(private_key, message: bytes) -> bytes:
    signature = private_key.sign(
        message,
        padding.PSS(
            mgf=padding.MGF1(hashes.SHA256()),
            salt_length=padding.PSS.MAX_LENGTH,
        ),
        hashes.SHA256(),
    )
    return signature

def verify_signature(public_key, message: bytes, signature: bytes) -> bool:
    try:
        public_key.verify(
            signature,
            message,
            padding.PSS(
                mgf=padding.MGF1(hashes.SHA256()),
                salt_length=padding.PSS.MAX_LENGTH,
            ),
            hashes.SHA256(),
        )
        return True
    except Exception:
        return False

if __name__ == "__main__":
    print("=" * 70)
    print("STEP 1: Key generation (RSA-2048)")
    print("=" * 70)
    private_key, public_key = generate_keypair()

    pub_pem = public_key.public_bytes(
        encoding=serialization.Encoding.PEM,
        format=serialization.PublicFormat.SubjectPublicKeyInfo,
    ).decode()
    print("Public key (PEM, truncated):")
    print("\n".join(pub_pem.splitlines()[4]) + "\n...")

    message = b"Transfer INR 50,000 to Account #10293847 - authorized by RITHISH KUMAR D"
    print("\n" + "=" * 70)
    print("STEP 2: Sender signs the message with their PRIVATE key")
    print("=" * 70)
    print(f"Message: {message.decode()}")
    signature = sign_message(private_key, message)
    print(f"\nSignature (hex, first 64 chars): {signature.hex()[:64]}...")
    print(f"Signature length: {len(signature)} bytes")

    print("\n" + "=" * 70)
    print("STEP 3: Receiver verifies using sender's PUBLIC key")
    print("=" * 70)
```

```

is_valid = verify_signature(public_key, message, signature)
print(f"Verification result on ORIGINAL message: {is_valid}")

print("\n" + "=" * 70)
print("STEP 4: Tamper with the message and re-verify")
print("=" * 70)
tampered_message = b"Transfer INR 90,000 to Account #10293847 - authorized by RITHISH KUMAR D"
print(f"Tampered message: {tampered_message.decode()}")
is_valid_tampered = verify_signature(public_key, tampered_message, signature)
print(f"Verification result on TAMPERED message: {is_valid_tampered}")

print("\n" + "=" * 70)
print("STEP 5: Verify with the WRONG public key (impersonation attempt)")
print("=" * 70)
_, imposter_public_key = generate_keypair()
is_valid_imposter = verify_signature(imposter_public_key, message, signature)
print(f"Verification result with a different key pair: {is_valid_imposter}")

print("\n" + "=" * 70)
print("ANALYSIS")
print("=" * 70)
print("""
1. Authentication: Only the holder of the private key could have produced
a signature that verifies correctly under the corresponding public key.
Step 5 shows that a signature made with one key pair fails verification
under a different public key, confirming that verification authenticates
the specific sender.

2. Integrity: Step 4 shows that any change to the signed message, however
small, causes verification to fail, because the signature is computed
over a SHA-256 hash of the message and RSA-PSS padding is deterministic
about matching content to signature.

3. Non-repudiation: Because only the sender possesses the private key, the
sender cannot later deny having signed the message (assuming the private
key was not compromised) - this is the property conventional MACs
(shared-secret schemes) cannot offer, since with a MAC either party
holding the shared key could have produced it.

4. Real-world scenario: This mirrors how digital signatures secure software
updates, financial transactions, and signed documents/PDF/e-invoices -
the signer's private key stays secret, while the public key can be
freely distributed (e.g., in an X.509 certificate) for anyone to verify
authenticity and integrity of the signed data.
""")

```

Output:

```

=====
STEP 1: Key generation (RSA-2048)
=====
Public key (PEM, truncated):
-----BEGIN PUBLIC KEY-----
MIIBIjANBgkqhkiG9w0BAQEFAAOCAQ8AMIIBCgKCAQEAn22YQ0ouxptHaHydLR/5
Hya/aqiDiJxtXJMocLvZgDD6Uj/v8htAKp631dmtHNCw9Ts4M+HgXRj0dpjp6zs3
ZsRuBYN5aldZ/7acXbCpFvUzLff3ATTQrUCnq1RiZ9QGGuMG9EACJ6v2lz4GU3q5a
...

=====
STEP 2: Sender signs the message with their PRIVATE key
=====
Message: Transfer INR 50,000 to Account #10293847 - authorized by RITHISH KUMAR D

Signature (hex, first 64 chars): 1a626d4bf9641f22c2935b3d30d63c019addfcb19d1ce0cd351be3efcb972c8...
Signature length: 256 bytes

=====
STEP 3: Receiver verifies using sender's PUBLIC key
=====
Verification result on ORIGINAL message: True

=====
STEP 4: Tamper with the message and re-verify
=====
Tampered message: Transfer INR 90,000 to Account #10293847 - authorized by RITHISH KUMAR D
Verification result on TAMPERED message: False

=====
STEP 5: Verify with the WRONG public key (impersonation attempt)

```

```
=====
Verification result with a different key pair: False
=====
```

```
=====
ANALYSIS
=====
```

1. Authentication: Only the holder of the private key could have produced a signature that verifies correctly under the corresponding public key. Step 5 shows that a signature made with one key pair fails verification under a different public key, confirming that verification authenticates the specific sender.
2. Integrity: Step 4 shows that any change to the signed message, however small, causes verification to fail, because the signature is computed over a SHA-256 hash of the message and RSA-PSS padding is deterministic about matching content to signature.
3. Non-repudiation: Because only the sender possesses the private key, the sender cannot later deny having signed the message (assuming the private key was not compromised) - this is the property conventional MACs (shared-secret schemes) cannot offer, since with a MAC either party holding the shared key could have produced it.
4. Real-world scenario: This mirrors how digital signatures secure software updates, financial transactions, and signed documents/PDF/e-invoices - the signer's private key stays secret, while the public key can be freely distributed (e.g., in an X.509 certificate) for anyone to verify authenticity and integrity of the signed data.

Question 4

Create a simplified Kerberos authentication simulation in C/Python that includes ticket generation, distribution, and validation. Analyze the workflow and evaluate how replay attacks are prevented in your implementation.

Program (Python):

```
"""
Experiment 4: Simplified Kerberos authentication simulation.
Includes ticket generation, distribution, validation and demonstrates
how replay attacks are prevented using timestamps + nonces.
"""

import os
import time
import hashlib
import hmac
import json

def derive_key(secret: str) -> bytes:
    return hashlib.sha256(secret.encode()).digest()

class KDC:
    """Key Distribution Center: holds long-term keys for Client and Server,
    issues a Ticket Granting Ticket (TGT) and later a Service Ticket."""

    def __init__(self):
        self.client_keys = {} # client_id -> key
        self.service_keys = {} # service_id -> key
        self.used_nonces = set() # replay-attack protection

    def register_client(self, client_id: str, password: str):
        self.client_keys[client_id] = derive_key(password)

    def register_service(self, service_id: str, secret: str):
        self.service_keys[service_id] = derive_key(secret)

    def issue_ticket(self, client_id: str, service_id: str, validity_seconds: int = 30):
        """AS + TGS combined for simplicity: issue a service ticket directly,
        encrypted (HMAC-sealed) with the service's key, plus a session key
        sealed with the client's key."""
        if client_id not in self.client_keys:
            raise ValueError("Unknown client")
        if service_id not in self.service_keys:
            raise ValueError("Unknown service")

        session_key = os.urandom(16)
        issue_time = time.time()
        nonce = os.urandom(8).hex()

        ticket = {
            "client_id": client_id,
            "service_id": service_id,
            "session_key": session_key.hex(),
            "issue_time": issue_time,
            "expiry_time": issue_time + validity_seconds,
            "nonce": nonce,
        }
        ticket_bytes = json.dumps(ticket, sort_keys=True).encode()
        mac = hmac.new(self.service_keys[service_id], ticket_bytes, hashlib.sha256).hexdigest()
        sealed_ticket = {"data": ticket, "mac": mac}

        # session key delivered to client sealed under the client's own key
        client_key = self.client_keys[client_id]
        session_key_mac = hmac.new(client_key, session_key, hashlib.sha256).hexdigest()

        return sealed_ticket, session_key, session_key_mac

class Service:
    """The application server that validates tickets presented by clients."""

    def __init__(self, service_id: str, kdc: KDC):
        self.service_id = service_id
```

```

self.kdc = kdc
self.seen_nonces = set() # replay protection at the service side

def validate_ticket(self, sealed_ticket: dict, presented_authenticator_time: float):
    ticket = sealed_ticket["data"]
    mac = sealed_ticket["mac"]

    # 1. Integrity check: recompute MAC using the service's own secret key
    service_key = self.kdc.service_keys[self.service_id]
    ticket_bytes = json.dumps(ticket, sort_keys=True).encode()
    expected_mac = hmac.new(service_key, ticket_bytes, hashlib.sha256).hexdigest()
    if not hmac.compare_digest(mac, expected_mac):
        return False, "Ticket integrity check FAILED (invalid MAC)"

    # 2. Expiry check
    now = time.time()
    if now > ticket["expiry_time"]:
        return False, "Ticket EXPIRED"

    # 3. Replay protection: reject a nonce that has already been used
    if ticket["nonce"] in self.seen_nonces:
        return False, "REPLAY DETECTED: nonce already used"
    self.seen_nonces.add(ticket["nonce"])

    # 4. Authenticator freshness (client must prove it holds the session
    # key by sending a fresh timestamp close to "now")
    if abs(now - presented_authenticator_time) > 5:
        return False, "Authenticator timestamp too far from current time"

    return True, f"Access GRANTED to {ticket['client_id']} for {ticket['service_id']}"

if __name__ == "__main__":
    print("=" * 70)
    print("STEP 1: Setup KDC, register client and service")
    print("=" * 70)
    kdc = KDC()
    kdc.register_client("rithish_kumar", "client_password_123")
    kdc.register_service("file_server", "service_secret_456")
    file_service = Service("file_server", kdc)
    print("Client 'rithish_kumar' and Service 'file_server' registered with the KDC.")

    print("\n" + "=" * 70)
    print("STEP 2: KDC issues a ticket (valid for 30 seconds)")
    print("=" * 70)
    ticket, session_key, session_key_mac = kdc.issue_ticket("rithish_kumar", "file_server")
    print(f"Ticket issued for client: {ticket['data']['client_id']}")
    print(f"Session key (hex) : {session_key.hex()}")
    print(f"Ticket nonce : {ticket['data']['nonce']}")
    print(f"Ticket expiry (unix) : {ticket['data']['expiry_time']:.2f}")

    print("\n" + "=" * 70)
    print("STEP 3: Client presents the ticket to the service (valid request)")
    print("=" * 70)
    ok, msg = file_service.validate_ticket(ticket, presented_authenticator_time=time.time())
    print(f"Result: {msg}")

    print("\n" + "=" * 70)
    print("STEP 4: Attacker REPLAYS the exact same ticket")
    print("=" * 70)
    ok_replay, msg_replay = file_service.validate_ticket(ticket, presented_authenticator_time=time.time())
    print(f"Result: {msg_replay}")

    print("\n" + "=" * 70)
    print("STEP 5: A tampered ticket (client_id altered) is rejected")
    print("=" * 70)
    forged_ticket = json.loads(json.dumps(ticket)) # deep copy
    forged_ticket["data"]["client_id"] = "attacker"
    ok_forged, msg_forged = file_service.validate_ticket(forged_ticket, presented_authenticator_time=time.time())
    print(f"Result: {msg_forged}")

    print("\n" + "=" * 70)
    print("STEP 6: An expired ticket is rejected")
    print("=" * 70)
    expired_ticket, _, _ = kdc.issue_ticket("rithish_kumar", "file_server", validity_seconds=-1)
    ok_exp, msg_exp = file_service.validate_ticket(expired_ticket, presented_authenticator_time=time.time())
    print(f"Result: {msg_exp}")

    print("\n" + "=" * 70)
    print("ANALYSIS")

```

```

    print("=" * 70)
    print("""
1. Ticket generation & distribution: The KDC (playing the combined role of
Authentication Server + Ticket Granting Server) issues a ticket bound to
a specific client and service, sealed with an HMAC computed under the
service's long-term key, plus a fresh random session key delivered
securely to the client.

2. Ticket validation: The service recomputes the HMAC to confirm the ticket
was genuinely issued by the trusted KDC and has not been altered
(Step 5), and checks the expiry timestamp (Step 6) to limit the ticket's
window of usefulness.

3. Replay-attack prevention: Each ticket carries a unique, single-use nonce.
The service tracks nonces it has already accepted (seen_nonces) and
rejects any ticket reusing one (Step 4). Combined with short ticket
lifetimes and an authenticator timestamp that must be close to the
current time, this prevents an attacker who intercepts a ticket from
successfully re-submitting it later, since a real Kerberos deployment
layers: (a) nonce/cache tracking, (b) tight clock-skew tolerance, and
(c) short-lived tickets, all of which this simulation demonstrates.
""")

```

Output:

```

=====
STEP 1: Setup KDC, register client and service
=====
Client 'rithish_kumar' and Service 'file_server' registered with the KDC.

=====
STEP 2: KDC issues a ticket (valid for 30 seconds)
=====
Ticket issued for client: rithish_kumar
Session key (hex)       : 66a5ef9b51a577a52ff57db9e14c7df9
Ticket nonce            : 6cfd507bd48f27b2
Ticket expiry (unix)    : 1786288871.09

=====
STEP 3: Client presents the ticket to the service (valid request)
=====
Result: Access GRANTED to rithish_kumar for file_server

=====
STEP 4: Attacker REPLAYS the exact same ticket
=====
Result: REPLAY DETECTED: nonce already used

=====
STEP 5: A tampered ticket (client_id altered) is rejected
=====
Result: Ticket integrity check FAILED (invalid MAC)

=====
STEP 6: An expired ticket is rejected
=====
Result: Ticket EXPIRED

=====
ANALYSIS
=====

1. Ticket generation & distribution: The KDC (playing the combined role of
Authentication Server + Ticket Granting Server) issues a ticket bound to
a specific client and service, sealed with an HMAC computed under the
service's long-term key, plus a fresh random session key delivered
securely to the client.

2. Ticket validation: The service recomputes the HMAC to confirm the ticket
was genuinely issued by the trusted KDC and has not been altered
(Step 5), and checks the expiry timestamp (Step 6) to limit the ticket's
window of usefulness.

3. Replay-attack prevention: Each ticket carries a unique, single-use nonce.
The service tracks nonces it has already accepted (seen_nonces) and
rejects any ticket reusing one (Step 4). Combined with short ticket
lifetimes and an authenticator timestamp that must be close to the
current time, this prevents an attacker who intercepts a ticket from
successfully re-submitting it later, since a real Kerberos deployment

```

layers: (a) nonce/cache tracking, (b) tight clock-skew tolerance, and (c) short-lived tickets, all of which this simulation demonstrates.

Question 5

Develop a C/Python program to parse and validate X.509 certificates. Analyze certificate fields (issuer, validity, public key) and evaluate the trustworthiness of certificates based on validation checks.

Program (Python):

```
"""
Experiment 5: Parse and validate X.509 certificates.
Analyze certificate fields (issuer, validity, public key) and evaluate
trustworthiness based on validation checks.
"""

import datetime
from cryptography import x509
from cryptography.x509.oid import NameOID
from cryptography.hazmat.primitives import hashes
from cryptography.hazmat.primitives.asymmetric import rsa
from cryptography.hazmat.primitives import serialization

def build_self_signed_ca(common_name="SIMATS Root CA", days_valid=365):
    """Create a self-signed CA certificate (acts as our trusted root)."""
    key = rsa.generate_private_key(public_exponent=65537, key_size=2048)
    subject = issuer = x509.Name([
        x509.NameAttribute(NameOID.COUNTRY_NAME, "IN"),
        x509.NameAttribute(NameOID.ORGANIZATION_NAME, "SIMATS Engineering"),
        x509.NameAttribute(NameOID.COMMON_NAME, common_name),
    ])
    now = datetime.datetime.utcnow()
    cert = (
        x509.CertificateBuilder()
        .subject_name(subject)
        .issuer_name(issuer)
        .public_key(key.public_key())
        .serial_number(x509.random_serial_number())
        .not_valid_before(now - datetime.timedelta(days=1))
        .not_valid_after(now + datetime.timedelta(days=days_valid))
        .add_extension(x509.BasicConstraints(ca=True, path_length=None), critical=True)
        .sign(key, hashes.SHA256())
    )
    return key, cert

def build_leaf_cert(ca_key, ca_cert, common_name="www.simats-portal.edu.in", days_valid=90, backdated=False):
    """Create a leaf certificate signed by the CA."""
    key = rsa.generate_private_key(public_exponent=65537, key_size=2048)
    subject = x509.Name([
        x509.NameAttribute(NameOID.COUNTRY_NAME, "IN"),
        x509.NameAttribute(NameOID.ORGANIZATION_NAME, "SIMATS Engineering"),
        x509.NameAttribute(NameOID.COMMON_NAME, common_name),
    ])
    now = datetime.datetime.utcnow()
    not_before = now - datetime.timedelta(days=800) if backdated else now - datetime.timedelta(hours=1)
    not_after = not_before + datetime.timedelta(days=days_valid)
    cert = (
        x509.CertificateBuilder()
        .subject_name(subject)
        .issuer_name(ca_cert.subject)
        .public_key(key.public_key())
        .serial_number(x509.random_serial_number())
        .not_valid_before(not_before)
        .not_valid_after(not_after)
        .add_extension(x509.BasicConstraints(ca=False, path_length=None), critical=True)
        .sign(ca_key, hashes.SHA256())
    )
    return key, cert

def print_cert_fields(cert: x509.Certificate, label: str):
    print(f"--- {label} ---")
    print(f"Subject      : {cert.subject.rfc4514_string()}")
    print(f"Issuer       : {cert.issuer.rfc4514_string()}")
    print(f"Serial Number : {cert.serial_number}")
    print(f"Not Before    : {cert.not_valid_before_utc}")
    print(f"Not After     : {cert.not_valid_after_utc}")
    pub = cert.public_key()
```

```

print(f"Public Key      : RSA-{pub.key_size} bits")
print(f"Signature Algo: {cert.signature_hash_algorithm.name}")

def validate_certificate(cert: x509.Certificate, ca_cert: x509.Certificate):
    """Run a small set of trust checks: signature validity, expiry window,
    and issuer match against the trusted CA."""
    checks = {}

    # 1. Validity period check
    now = datetime.datetime.now(datetime.timezone.utc)
    checks["within_validity_period"] = cert.not_valid_before_utc <= now <= cert.not_valid_after_utc

    # 2. Issuer matches our trusted CA's subject
    checks["issuer_matches_trusted_ca"] = cert.issuer == ca_cert.subject

    # 3. Signature verification: the CA's public key must verify the cert's signature
    try:
        ca_cert.public_key().verify(
            cert.signature,
            cert.tbs_certificate_bytes,
            __import__("cryptography.hazmat.primitives.asymmetric.padding", fromlist=["PKCS1v15"]).PKCS1v15(),
            cert.signature_hash_algorithm,
        )
        checks["signature_verified_by_ca"] = True
    except Exception:
        checks["signature_verified_by_ca"] = False

    # 4. Reasonable validity duration (flag suspiciously long-lived certs)
    duration_days = (cert.not_valid_after_utc - cert.not_valid_before_utc).days
    checks["validity_duration_reasonable(<=825d)"] = duration_days <= 825

    overall_trusted = all(checks.values())
    return checks, overall_trusted

if __name__ == "__main__":
    print("=" * 70)
    print("STEP 1: Build a trusted Root CA and a legitimate leaf certificate")
    print("=" * 70)
    ca_key, ca_cert = build_self_signed_ca()
    print_cert_fields(ca_cert, "Root CA Certificate")

    leaf_key, leaf_cert = build_leaf_cert(ca_key, ca_cert)
    print()
    print_cert_fields(leaf_cert, "Leaf Certificate (issued by CA)")

    print("\n" + "=" * 70)
    print("STEP 2: Validate the LEGITIMATE leaf certificate")
    print("=" * 70)
    checks, trusted = validate_certificate(leaf_cert, ca_cert)
    for check, result in checks.items():
        print(f"  {check:38s}: {'PASS' if result else 'FAIL'}")
    print(f"\nOverall trust decision: {'TRUSTED' if trusted else 'NOT TRUSTED'}")

    print("\n" + "=" * 70)
    print("STEP 3: Validate a certificate signed by an UNTRUSTED CA")
    print("=" * 70)
    rogue_ca_key, rogue_ca_cert = build_self_signed_ca(common_name="Rogue CA")
    rogue_leaf_key, rogue_leaf_cert = build_leaf_cert(rogue_ca_key, rogue_ca_cert)
    checks_rogue, trusted_rogue = validate_certificate(rogue_leaf_cert, ca_cert)
    for check, result in checks_rogue.items():
        print(f"  {check:38s}: {'PASS' if result else 'FAIL'}")
    print(f"\nOverall trust decision: {'TRUSTED' if trusted_rogue else 'NOT TRUSTED'}")

    print("\n" + "=" * 70)
    print("STEP 4: Validate an EXPIRED / backdated certificate")
    print("=" * 70)
    _, expired_cert = build_leaf_cert(ca_key, ca_cert, days_valid=90, backdated=True)
    checks_exp, trusted_exp = validate_certificate(expired_cert, ca_cert)
    for check, result in checks_exp.items():
        print(f"  {check:38s}: {'PASS' if result else 'FAIL'}")
    print(f"\nOverall trust decision: {'TRUSTED' if trusted_exp else 'NOT TRUSTED'}")

    print("\n" + "=" * 70)
    print("ANALYSIS")
    print("=" * 70)
    print("""
1. Certificate fields: Subject identifies who the certificate is issued to,
Issuer identifies the CA that vouches for it, the validity window

```

(Not Before / Not After) bounds its trusted lifetime, and the embedded public key is what relying parties use to verify the certificate holder's signatures or establish an encrypted channel.

2. Signature-based trust: A certificate is only trustworthy if it was signed by a CA the verifier already trusts. Step 3 shows that a certificate that is well-formed and unexpired is still REJECTED once the signature check shows it was not actually signed by our trusted root CA - this is what prevents an attacker from self-issuing a certificate claiming to be a legitimate site.
 3. Validity window enforcement: Step 4 shows a backdated/expired certificate is rejected on the "within_validity_period" check, and also flagged on the "reasonable duration" heuristic, since real CAs and browsers reject certificates whose lifetimes are outside policy limits (e.g., the CA/Browser Forum currently caps public TLS certs at 398 days).
 4. Overall trustworthiness evaluation therefore combines multiple independent checks (issuer identity, cryptographic signature validity, and time-based validity) rather than relying on any single field, which mirrors how real X.509 chain validation works in browsers and TLS libraries.
- ```
""")
```

## Output:

```
=====
STEP 1: Build a trusted Root CA and a legitimate leaf certificate
=====

--- Root CA Certificate ---
Subject : CN=SIMATS Root CA,O=SIMATS Engineering,C=IN
Issuer : CN=SIMATS Root CA,O=SIMATS Engineering,C=IN
Serial Number : 70054935737051817343141296427882068410759644365
Not Before : 2026-08-08 15:21:14+00:00
Not After : 2027-08-09 15:21:14+00:00
Public Key : RSA-2048 bits
Signature Algo: sha256

--- Leaf Certificate (issued by CA) ---
Subject : CN=www.simats-portal.edu.in,O=SIMATS Engineering,C=IN
Issuer : CN=SIMATS Root CA,O=SIMATS Engineering,C=IN
Serial Number : 244835892179943696463755308802160576202584942774
Not Before : 2026-08-09 14:21:14+00:00
Not After : 2026-11-07 14:21:14+00:00
Public Key : RSA-2048 bits
Signature Algo: sha256

=====
STEP 2: Validate the LEGITIMATE leaf certificate
=====

within_validity_period : PASS
issuer_matches_trusted_ca : PASS
signature_verified_by_ca : PASS
validity_duration_reasonable(<=825d) : PASS

Overall trust decision: TRUSTED

=====
STEP 3: Validate a certificate signed by an UNTRUSTED CA
=====

within_validity_period : PASS
issuer_matches_trusted_ca : FAIL
signature_verified_by_ca : FAIL
validity_duration_reasonable(<=825d) : PASS

Overall trust decision: NOT TRUSTED

=====
STEP 4: Validate an EXPIRED / backdated certificate
=====

within_validity_period : FAIL
issuer_matches_trusted_ca : PASS
signature_verified_by_ca : PASS
validity_duration_reasonable(<=825d) : PASS

Overall trust decision: NOT TRUSTED

=====
ANALYSIS
```

- =====
1. Certificate fields: Subject identifies who the certificate is issued to, Issuer identifies the CA that vouches for it, the validity window (Not Before / Not After) bounds its trusted lifetime, and the embedded public key is what relying parties use to verify the certificate holder's signatures or establish an encrypted channel.
  2. Signature-based trust: A certificate is only trustworthy if it was signed by a CA the verifier already trusts. Step 3 shows that a certificate that is well-formed and unexpired is still REJECTED once the signature check shows it was not actually signed by our trusted root CA - this is what prevents an attacker from self-issuing a certificate claiming to be a legitimate site.
  3. Validity window enforcement: Step 4 shows a backdated/expired certificate is rejected on the "within\_validity\_period" check, and also flagged on the "reasonable duration" heuristic, since real CAs and browsers reject certificates whose lifetimes are outside policy limits (e.g., the CA/Browser Forum currently caps public TLS certs at 398 days).
  4. Overall trustworthiness evaluation therefore combines multiple independent checks (issuer identity, cryptographic signature validity, and time-based validity) rather than relying on any single field, which mirrors how real X.509 chain validation works in browsers and TLS libraries.

## RUBRICS FOR CALCULATION:

| Criteria                  | Weightage | Level 4 - Exemplary                                                                           | Level 3 - Proficient                                        | Level 2 - Developing                                | Level 1 - Beginning                              |
|---------------------------|-----------|-----------------------------------------------------------------------------------------------|-------------------------------------------------------------|-----------------------------------------------------|--------------------------------------------------|
| Understanding of Concepts | 25        | Demonstrates thorough understanding and effectively integrates multiple concepts/technologies | Good understanding with proper integration of most concepts | Basic understanding; limited or partial integration | Poor understanding; unable to integrate concepts |
| Experimental Design       | 30        | Well-planned, systematic implementation with optimal solution                                 | Correct implementation with minor issues                    | Basic implementation with errors or inefficiencies  | Incorrect or incomplete implementation           |
| Use of Tools              | 20        | Effective and advanced use of tools/technologies with proper configuration                    | Appropriate use of tools with correct execution             | Limited or inconsistent use of tools                | Incorrect or minimal use of tools                |
| Analysis of Results       | 15        | Insightful analysis with accurate interpretation and validation of results                    | Good analysis with reasonable interpretation                | Basic analysis with limited insights                | Poor or incorrect analysis of results            |
| Documentation             | 10        | Clear, well-structured documentation with diagrams, code, and results                         | Organized documentation with necessary details              | Basic documentation with missing details            | Poor or incomplete documentation                 |